ADVANCED MACHINE LEARNING + DEEP LEARNING PROJECT

Microclimate Modelling via Neural Surrogates and Computational Fluid Dynamics

Phase 1 — Machine Learning Evaluation

Project Report

Dataset: UrbanTALES — Idealized Building Arrays
Team / Project Code: T006

Submitted by

Team member	Student ID
Suhaani Garg	2401010462
Jashvita Lakshmi Omkaram	2401010310
Aarya Srivastava	2401010008
Palak	2401010042
Samarth Sangtani	2401010408

Repository
https://github.com/aarya-sr/T006-urban-microclimate-modelling

Report scope: All numerical results are taken from the saved outputs of notebooks ML_01–ML_07 (fresh run, identical model settings in ML_06 and ML_07). Phase 2 (spatial field prediction) is a proposed direction and is not reported as a result.

Abstract

This report documents Phase 1 (Machine Learning) of the project "Microclimate Modelling via Neural Surrogates and Computational Fluid Dynamics". Using the idealized building-array subset of the UrbanTALES computational fluid dynamics (CFD) dataset, we build a scalar surrogate that predicts the canopy-averaged wind speed (m/s) of a simulated case from layout-level geometric summaries and wind direction. The data comprise 224 simulation cases (112 unique layouts $\times$ two wind directions, 0° and 45°) grouped into 16 plan families.

The pipeline covers dataset selection, joining of the raw metadata and flow-data tables, auditing of the supplied profile, topology and NetCDF archives, data validation, exploratory analysis, feature engineering with a leakage audit, group-aware validation, and four models: a training-fold mean baseline, Ridge regression, Random Forest and XGBoost. The primary evaluation holds out entire plan families. Under family-held-out 5-fold cross-validation, XGBoost achieves an MAE of 0.232 m/s (RMSE 0.309 m/s, $R^2 = 0.903$), Random Forest 0.256 m/s, Ridge regression 0.400 m/s and the mean baseline 0.798 m/s. XGBoost reduces the baseline error by 70.9%, about 11% of the mean wind speed. The difference between XGBoost and Random Forest is not reliable: the paired family-bootstrap interval for the MAE difference (+0.024 m/s) spans zero. Holding out sibling layouts instead of families makes Random Forest look about four times better (MAE 0.065 m/s), which confirms that family-held-out validation is the honest generalisation estimate. Plan-area density and its square carry about 73% of the importance in both tree models.

The scalar surrogate predicts one canopy-averaged value per case and cannot represent local wake patterns or street-canyon effects; no temperature claims are made.

Contents

1 Introduction **3**

1.1 Background and Motivation . . . 3

1.2 Project Brief and Two-Phase Structure . . . 3

1.3 Problem Statement . . . 3

1.4 Research Questions . . . 3

1.5 Scope . . . 4

2 Literature Review **5**

2.1 Background Reading . . . 5

2.2 Why These Models . . . 5

3 Dataset **6**

3.1 Requirements and Choice of Dataset . . . 6

3.2 About UrbanTALES Idealized Arrays . . . 6

3.3 Raw Files Used . . . 6

4 Data Extraction and Loading **7**

4.1 Auditing the ZIP Archives . . . 7

5 Data Cleaning, Joining and Validation **8**

5.1 Building the Layout-Level Table . . . 8

5.2 Derived Identifiers . . . 8

5.3 Target Join . . . 8

5.4 Missing Values, Duplicates and Outliers . . . 8

5.5 Saved Outputs . . . 8

6 Exploratory Data Analysis and Target Definition **9**

6.1 Target Definition . . . 9

6.2 Findings . . . 9

7 Feature Engineering and Leakage Audit **11**

7.1 Engineered Features . . . 11

7.2 Final Predictor Sets . . . 11

7.3 Leakage Controls . . . 11

7.4 Redundancy Check . . . 12

8 Validation Design and Preprocessing 13

8.1 Why Grouped Validation . . . 13

8.2 Protocols . . . 13

8.3 Preprocessing Pipelines . . . 14

9 Modelling 15

9.1 Training-Fold Mean Baseline . . . 15

9.2 Ridge Regression (ML_05) . . . 15

9.3 Random Forest and XGBoost (ML_06) . . . 15

10 Evaluation Methodology 16

11 Results 17

11.1 Primary Result: Family-Held-Out 5-Fold CV . . . 17

11.2 Effect of Validation Protocol . . . 17

11.3 Feature Ablation (family_5fold) . . . 18

11.4 Ridge Regularisation Sensitivity . . . 18

11.5 Random Forest versus XGBoost . . . 18

11.6 Error Diagnostics and Feature Importance . . . 19

11.7 Discussion . . . 20

12 Implementation Notes, Issues and Reproducibility 22

12.1 Environment . . . 22

12.2 Issues Encountered and Fixes . . . 22

12.3 Run Order and Outputs . . . 23

13 Limitations 24

14 Conclusion and Phase 2 Outlook 25

15 Team Contributions 26

References 27

A Data Dictionary of the Modelling Table 28

1 Introduction

1.1 Background and Motivation

Wind conditions inside urban canopies govern pedestrian comfort, ventilation and heat exposure. CFD simulation resolves these flows well but is expensive, so only a small number of layouts can be simulated. A fast surrogate that predicts flow quantities from a layout description could allow many more configurations to be screened. The project brief frames the study as one about surrogate accuracy and uncertainty, not safety-critical design approval.

1.2 Project Brief and Two-Phase Structure

The brief defines a hybrid logic that moves from geometric summary features to spatial surrogate fields:

- **AML baseline (Phase 1, this report):** describe each layout with building density, height statistics and wind direction, and predict an area-averaged wind quantity using Random Forest or XGBoost.
- **Identified gap:** layout summaries cannot represent local wake patterns, street-canyon effects or detailed spatial variation.
- **DL model (Phase 2):** encode a rasterised layout and conditions as channels and train a compact U-Net or conditional CNN to predict a velocity field.
- **Fair comparison:** hold out entire layouts rather than random pixels or rows; report field MAE, normalised RMSE and error maps for unseen geometries.

1.3 Problem Statement

How well can layout-level geometric summaries and wind direction predict the canopy-averaged wind-speed scalar for idealized UrbanTALES cases, especially when an entire plan family is held out?

1.4 Research Questions

1. How are target values and geometry features distributed, and what data-quality limitations matter?
2. Do geometry summaries and wind-direction features carry predictive signal beyond a training-fold mean baseline?
3. Do nonlinear models improve over a regularised linear model under family-held-out evaluation?
4. How much do scores change when the split is made by family versus by layout?
5. Which claims are supported by a scalar target, and which spatial-flow claims remain out of scope?

1.5 Scope

Phase 1 evaluates a scalar baseline only. The raster-to-field U-Net or CNN is a Phase 2 direction and is not claimed to be implemented here. The target is one wind-speed value per simulation, not a velocity map, and the dataset is an airflow dataset, so no temperature claims are made.

2 Literature Review

2.1 Background Reading

The faculty brief lists two suggested pre-reads, and a peer-reviewed review of machine learning in fluid mechanics was added for context.

- **Goodfellow et al. (2014), Generative Adversarial Nets.** Introduces adversarial generative modelling. It is background on learning distributions; a GAN is not needed for this scalar regression task and is not used.
- **Farimani et al. (2017), Deep Learning for Wall Bounded Turbulent Flows.** Listed in the brief; it motivates data-driven modelling of flow behaviour. Full bibliographic metadata should be checked against the faculty-supplied reference, so no venue or DOI is invented here.
- **Brunton, Noack and Koumoutsakos (2020), Machine Learning for Fluid Mechanics,** *Annual Review of Fluid Mechanics* 52, 477–508. Places regression, reduced-order modelling and other ML methods in the fluid-mechanics context and stresses that model choice, physical knowledge and generalisation must be considered together.

Implication for Phase 1. Begin with transparent scalar regression baselines and a defensible validation protocol. The brief names Random Forest and XGBoost on geometric summaries as the AML baseline. Spatially structured CNN or U-Net learning requires paired spatial fields, aligned grids and field-level metrics, and belongs to the later phase.

2.2 Why These Models

Method	Role	What it tests	Limitation
Training-fold mean	No-feature reference	Whether features add value at all	Cannot represent layout or direction effects
Ridge regression	Linear model with L2 penalty	Whether mostly linear relationships suffice	Limited nonlinear interaction capacity
Random Forest	Bagged trees, feature subsampling	Nonlinear thresholds and interactions	Weak extrapolation; impurity importance can be biased
XGBoost	Sequential gradient-boosted trees	Whether boosting lowers error on tabular features	Hyperparameter-sensitive; importance is not causal

Table 2.1: Models compared and the role each plays.

Ridge minimises the residual sum of squares plus an L2 penalty, $\mathrm{RSS}+\alpha\sum\beta_j^2$, which discourages unstable large coefficients when predictors are correlated. Standardisation matters because the penalty otherwise acts unevenly on differently scaled variables.

3 Dataset

3.1 Requirements and Choice of Dataset

The project needs building geometry and CFD outputs, inlet conditions, layout identifiers that allow layout-wise splitting, and simulation metadata. The brief recommends UrbanTALES idealized building arrays as a CFD alternative to a self-generated OpenFOAM sweep.

Requirement	**Needed**	**Source**
Urban geometry	Shapes, spacing, heights	`metadata_ideal.json`
Simulation output	Wind quantity per case	`UrbanTALES-flowdata.csv`, column `U`
Input conditions	Wind direction	`metadata_ideal.json`
Layout identification	IDs to separate arrangements	Folder name; layout and family IDs derived from it
Metadata	Density class, configuration	`metadata_ideal.json`, `metadata.csv`

Table 3.1: Project requirements and where each is met.

3.2 About UrbanTALES Idealized Arrays

The idealized subset contains 224 simulated building-layout cases covering different building densities, aligned and staggered arrangements, building-height patterns and wind directions. It is an airflow dataset: a strong match for wind modelling but not a temperature dataset. The supplied extraction notes state that the data are provided under CC BY-NC 4.0; attribution must be retained.

3.3 Raw Files Used

File	**Content**	**Use**
`metadata_ideal.json`	Per-case geometry and wind direction	Defines the 224-case subset
`metadata.csv`	Full catalogue (538 cases)	Cross-check of idealized IDs
`UrbanTALES-flowdata.csv`	Flow statistics per case (538 rows, 41 columns)	Source of the target (`U`)
`selected_files.zip`	32 sampled vertical profiles	Diagnostic cross-check of the target
`selected_files (1).zip`	32 sampled topology grids	Layout-grid audit; Phase 2 candidate
`selected_files (2).zip`	32 sampled NetCDF files (16 cases)	Spatial-field audit; Phase 2 candidate

Table 3.2: Raw input files and their use.

4 Data Extraction and Loading

All extraction is implemented in notebook ML_01 and is reproducible from the raw files; the processed CSV and data dictionary are never edited by hand. Every notebook locates the repository root by walking up from the working directory until it finds both `data/raw` and the `Machine Learning Pipeline` folder. The idealized-case JSON, catalogue CSV and flow-data CSV are read independently and inspected before any join, and a clear error is raised if a required raw file is missing.

4.1 Auditing the ZIP Archives

Each archive is read in memory with `zipfile` and `BytesIO`, so the originals stay intact.

Archive	Files	Cases	Size (MB)
NetCDF	32	16	64.0
Profiles	32	32	2.4
Topology	32	32	15.0

Table 4.1: Raw archive coverage (ML_01 audit).

- **Profiles:** each CSV (columns z, U) is integrated from the ground to the mean building height with the trapezoidal rule and divided by the mean height. Compared with the flow-table `U` for the sampled cases, the mean absolute difference is **0.047 m/s**, which supports the target definition (diagnostic only; the flow-table value remains the target).
- **Topology:** each member is parsed as a numeric grid (e.g. 384×640 for the sparsest layout, 240×400 for a denser one) with building cells at 0 or 16 m.
- **NetCDF:** files are opened with `h5py` (NetCDF4/HDF5) and, if that fails, with `scipy.io.netcdf_file` (classic NetCDF). The archive mixes both formats.

5 Data Cleaning, Joining and Validation

5.1 Building the Layout-Level Table

Each entry in `metadata_ideal.json` becomes one row. The simulation ID is the folder name; the layout ID removes the wind-direction suffix; wind direction and plan-area density are read from the metadata. The unit of observation is one layout under one wind direction.

5.2 Derived Identifiers

- **Family ID:** extracted from the layout ID with the pattern `^[UV]([AS]\d+)`, i.e. the arrangement letter (A aligned, S staggered) and plan-area-density code. This gives 16 families used as the primary validation groups (14 cases, 7 layouts each).
- **Height pattern:** "uniform" if the layout ID starts with U, otherwise the code after the first hyphen (CL, HR or SM), stored as `configuration`.
- The catalogue label (`Config`) is kept as `configuration_metadata_label` for traceability but excluded from predictors, because the CL/SM labels are inconsistent in the catalogue; the folder-code configuration is used instead.

5.3 Target Join

The target is joined by simulation ID (`NameE`) with a one-to-one validation, never by row order. The notebook stops if there are duplicate IDs, a missing `U`, an idealized ID absent from the catalogue, a row count other than 224, layouts other than 112, families other than 16, any missing cell, or wind directions other than $\{0, 45\}$. All checks passed.

5.4 Missing Values, Duplicates and Outliers

The table has 224 rows and 19 columns, no missing or infinite cells, no duplicate IDs and a finite target for every case. No rows are removed. The 1.5×IQR rule flags 16 rows in `height_max_m` for diagnosis only, because unusual CFD outputs may be physically meaningful and must stay traceable.

5.5 Saved Outputs

ML_01 writes `layout_level_dataset.csv`, `data_dictionary.csv` (also copied to `docs/`), `dataset_summary.j`
and the archive audit tables (`raw_archive_manifest.csv`, `profile_target_crosscheck.csv`, `topology_archive_manifest.csv`, `netcdf_archive_manifest.csv`).

6 Exploratory Data Analysis and Target Definition

6.1 Target Definition

The target `target_U_canopy_mean_m_s` is the `U` column of the flow-data table: a canopy-averaged wind speed in m/s with one scalar per simulation. It is not a spatial velocity map.

6.2 Findings

Quantity	Value
Target range	0.899–5.224 m/s
Mean / median / SD	2.050 / 1.603 / 0.997 m/s (skewness 1.345)
Mean by wind direction	2.237 m/s at 0°; 1.862 m/s at 45°
Mean by density class	sparse 3.036; medium 1.595; dense 1.366 m/s
Mean by horizontal configuration	aligned 2.238; staggered 1.861 m/s
Paired within-layout change (45° − 0°)	mean −0.375 m/s, median −0.220 m/s (112 layouts)
Strongest Pearson correlation	plan-area density, $r = -0.747$
Family means	span 1.184–4.230 m/s across 16 families

Table 6.1: Key EDA findings (ML_02).

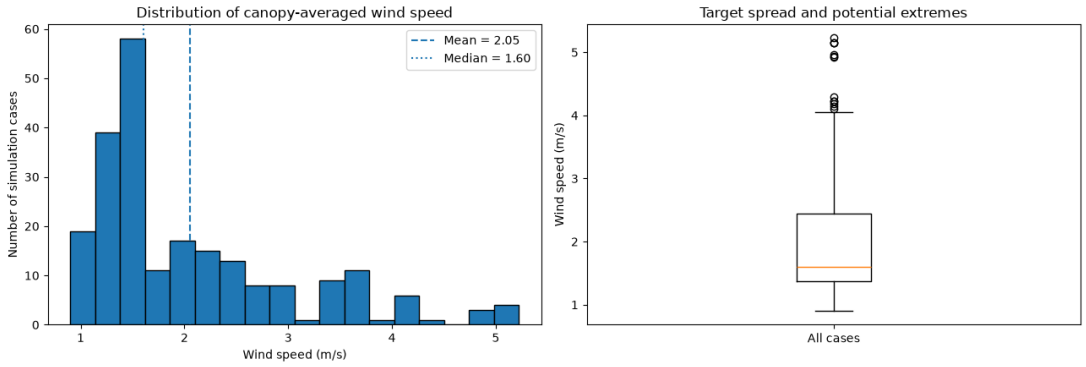


Figure 6.1: Distribution of the canopy-averaged wind speed: right-skewed, mean 2.05 and median 1.60 m/s.

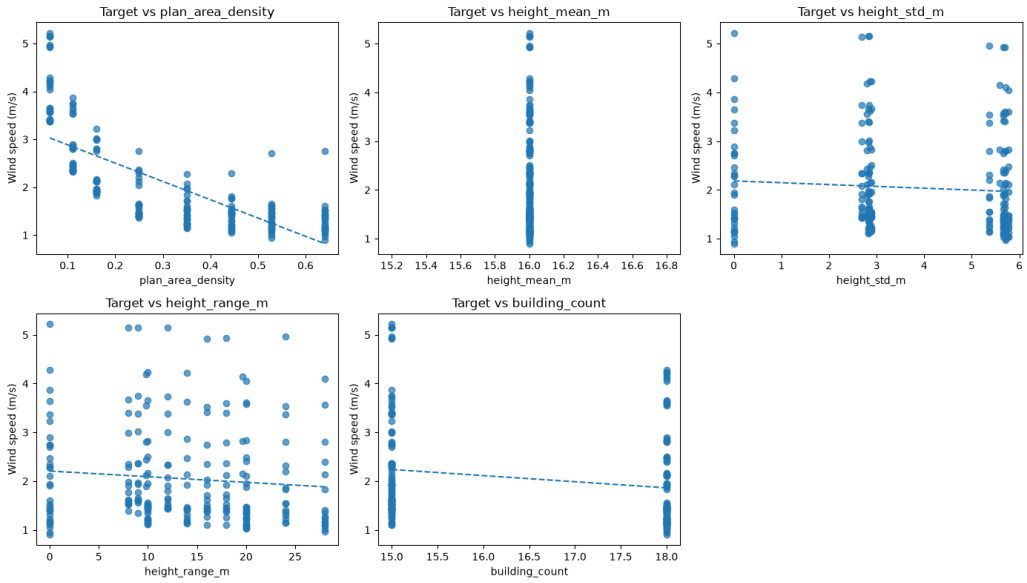


Figure 6.2: Target versus geometry features. Wind speed falls with plan-area density; height statistics show little linear trend.

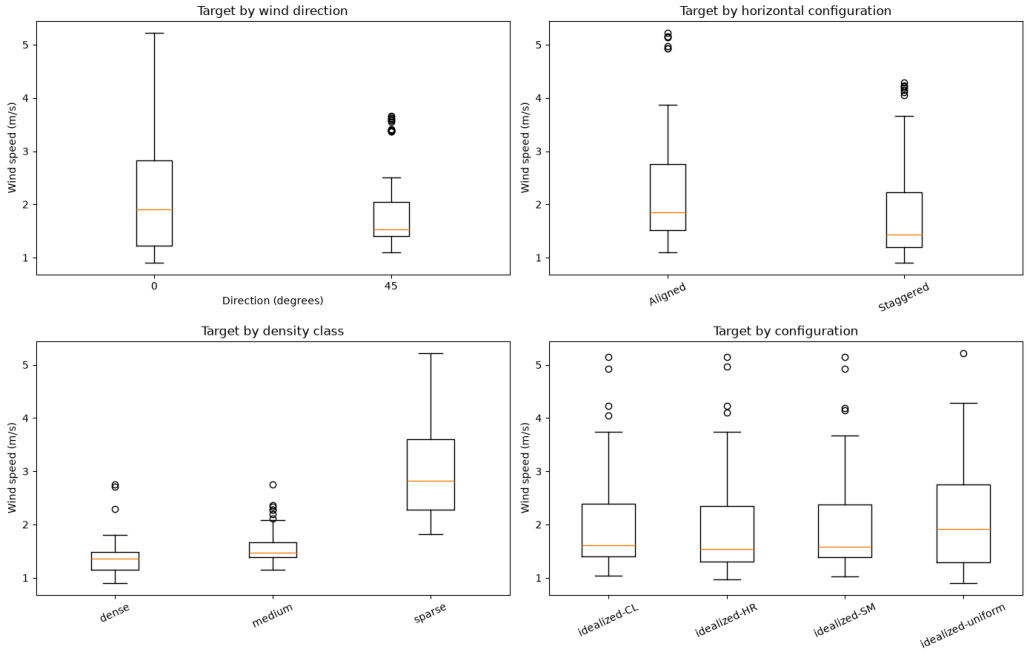


Figure 6.3: Target by wind direction, horizontal configuration, density class and height pattern.

Wind speed is strongly tied to plan-area density, is higher at 0° than at 45° and for aligned layouts, and is largely insensitive to the height pattern. The wide spread of family means shows why random row splits would be misleading. These are descriptive associations, not causal effects.

7 Feature Engineering and Leakage Audit

Feature engineering expresses plausible geometry and condition information; it cannot add CFD information or restore spatial detail lost by aggregation.

7.1 Engineered Features

Feature	Definition	Rationale
`plan_area_density_sq`	density^2	Simple nonlinear density response
`wind_direction_sin/cos`	sin, cos of direction	Circular variable; $\sin^2 + \cos^2 = 1$ asserted
`height_coefficient_variation`	height std / height mean	Relative height dispersion
`height_std, max, min, range, skew`	Statistics of the height list	Vertical heterogeneity
`frac_taller_than_mean`	Share of buildings above the mean	Distribution of tall elements

Table 7.1: Engineered features and rationale.

7.2 Final Predictor Sets

Primary engineered set (14 predictors): 11 numeric (`plan_area_density`, `plan_area_density_sq`, `height_std_m`, `height_max_m`, `height_min_m`, `height_range_m`, `height_skew`, `frac_taller_than_mean`, `wind_direction_sin`, `wind_direction_cos`, `height_coefficient_variation`) and 3 categorical (`configuration`, `horizontal_configuration`, `density_class`; one-hot encoded).

Baseline ablation set (9 predictors): `plan_area_density`, `height_std_m`, `height_max_m`, `building_count`, `wind_direction_sin`, `wind_direction_cos` plus the same three categoricals.

7.3 Leakage Controls

- An assertion confirms that no forbidden field is a predictor: the target, `simulation_id`, `layout_id`, `family_id`, `configuration_metadata_label` and the raw `wind_direction_deg`.
- Identifiers are used only to build held-out groups.
- `height_mean_m` (constant 16 m) and `building_count` are excluded from the primary set; `building_count` appears only in the baseline ablation.
- All learned preprocessing statistics (imputation, scaling, encoding) are fitted on training folds only.

7.4 Redundancy Check

Feature 1	**Feature 2**	$\lvert r \rvert$
plan_area_density	plan_area_density_sq	0.975
height_std_m	height_min_m	0.914
height_std_m	height_range_m	0.925
height_std_m	height_coefficient_variation	1.000
height_max_m	height_range_m	0.927
height_min_m	height_coefficient_variation	0.914
height_range_m	height_coefficient_variation	0.925
wind_direction_sin	wind_direction_cos	1.000

Table 7.2: Numeric pairs with $|r| \geq 0.90$ (ML_03). Kept for review; tree models tolerate redundancy and Ridge is regularised.

Because only 0° and 45° occur, sine and cosine are perfectly redundant and act as a two-state direction flag; no claim is made about other angles.

8 Validation Design and Preprocessing

8.1 Why Grouped Validation

Each layout appears twice (two directions) and layouts share a plan family. If sibling cases fall on both sides of a split, scores look better than performance on an unseen family. Group-aware splits are therefore used throughout.

8.2 Protocols

Protocol	Splitter and group	Role	Folds
`family_5fold`	GroupKFold(5) on `family_id`	Primary generalisation estimate	5
`family_loo`	LeaveOneGroupOut on `family_id`	Stricter secondary check	16
`layout_5fold`	GroupKFold(5) on `layout_id`	Interpolation-like check	5

Table 8.1: Validation protocols.

Fold	Train cases	Test cases	Train families	Test families
1	168	56	12	4
2–5	182	42	13	3

Table 8.2: family_5fold manifest (ML_04). A leakage check confirms no family appears in both train and test.

In `layout_5fold` every fold has test layouts (22–23 each) whose families are also present in training (16 shared families per fold), so it is treated as a secondary interpolation protocol.

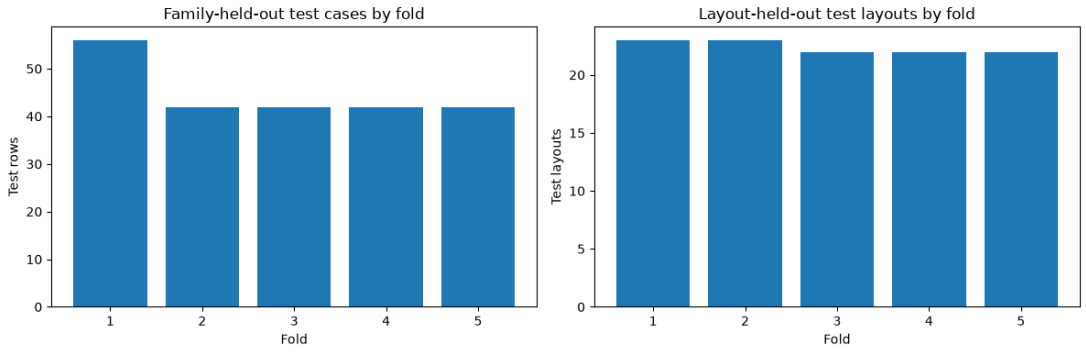


Figure 8.1: Fold sizes under the validation protocols (ML_04).

8.3 Preprocessing Pipelines

Model type	Numeric	Categorical
Ridge	Median imputation, StandardScaler	Most-frequent imputation, one-hot (`handle_unknown="ignore"`)
Random Forest, XGBoost	Median imputation only	Same as Ridge

Table 8.3: Preprocessing by model type.

Everything sits inside a scikit-learn `Pipeline` and `ColumnTransformer`, so `fit()` sees training rows only. In ML_04 the fitted preprocessor produces a 20-column matrix from the 14 predictors.

9 Modelling

9.1 Training-Fold Mean Baseline

Predicts the training-fold mean target for every test row. A model that cannot beat this adds no value.

9.2 Ridge Regression (ML_05)

Ridge with $\alpha = 1$ is the documented setting. A prespecified sensitivity analysis over $\alpha \in \{0.01, 0.1, 1, 10, 100\}$ under the same family folds is reported in Section 11.4; it is not nested hyperparameter optimisation.

9.3 Random Forest and XGBoost (ML_06)

	Random Forest	**XGBoost**
Trees	300	200
Depth / leaf	`min_samples_leaf`=2	`max_depth`=2, `min_child_weight`=3
Sampling	`max_features`=0.9	`subsample`=0.85, `colsample_bytree`=0.9
Learning	—	`learning_rate`=0.03, `reg_lambda`=2.0
Objective	Squared error	`reg:squarederror`, histogram trees
Seed	`random_state`=42	`random_state`=42

Table 9.1: Fixed model settings, shared by ML_06 and ML_07.

Hyperparameters are fixed, reproducible baseline settings; no search was run. Predictions are always out-of-fold: each case is predicted exactly once by a model that never saw its family (asserted). Tree importances are mapped back to original feature names, with one-hot columns *summed* under their categorical parent within each fold and then averaged across folds. Importance is descriptive, not causal.

10 Evaluation Methodology

All comparison is implemented in notebook ML_07.

Metric	Formula	Interpretation
MAE	mean $\lvert y - \hat{y} \rvert$	Typical absolute error in m/s; primary ranking metric
RMSE	$\sqrt{\text{mean}(y - \hat{y})^2}$	Penalises large misses
Normalised RMSE	RMSE / (max y− min y)	Scale-relative; target range 4.325 m/s
R^2	$1 - SS_{res}/SS_{tot}$	Can be negative; not an accuracy percentage

Table 10.1: Evaluation metrics.

Experiments: all four models under all three protocols; a feature ablation (baseline versus engineered set) under `family_5fold`; a family-clustered bootstrap (1,000 resamples, seed 42) for 95% MAE intervals; a paired Random Forest versus XGBoost comparison (per-case absolute-error difference, family bootstrap, 1,000 resamples, seed 0); and diagnostics (comparison chart, observed versus predicted, error by direction, importance). The intervals do not remove the uncertainty caused by having only 16 families.

11 Results

11.1 Primary Result: Family-Held-Out 5-Fold CV

Model	MAE (m/s)	RMSE (m/s)	Norm. RMSE	R^2	MAE 95% CI
XGBoost	**0.232**	**0.309**	**0.071**	**0.903**	0.182–0.288
Random Forest	0.256	0.380	0.088	0.854	0.176–0.337
Ridge Regression	0.400	0.507	0.117	0.740	0.312–0.500
Mean baseline	0.798	1.011	0.234	−0.033	0.566–1.082

Table 11.1: Primary comparison under family-held-out 5-fold CV (224 cases, 16 families).

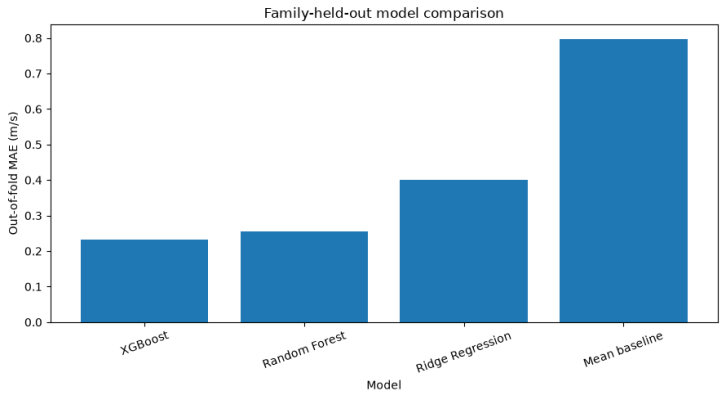


Figure 11.1: Out-of-fold MAE by model under family-held-out evaluation.

XGBoost reduces MAE by 0.565 m/s (70.9%) relative to the mean baseline, about 11% of the mean target value. Ridge regression, which uses the same features, already removes half the baseline error, so geometry summaries carry real predictive signal.

11.2 Effect of Validation Protocol

Model	family_5fold	family_loo	layout_5fold
XGBoost	0.232	0.238	0.150
Random Forest	0.256	0.250	0.065
Ridge Regression	0.400	0.403	0.332
Mean baseline	0.798	0.836	0.790

Table 11.2: MAE (m/s) by validation protocol.

Leave-one-family-out agrees with the 5-fold family result. Holding out sibling layouts instead of families makes Random Forest look about four times better (0.065 versus 0.256 m/s), because re-

lated layouts remain in training. The family-held-out value is therefore the honest generalisation estimate.

11.3 Feature Ablation (family_5fold)

Model	Baseline features	Engineered features	Change
Ridge Regression	0.440	0.400	−0.039 (improves)
Random Forest	0.240	0.256	+0.017 (worsens)
XGBoost	0.233	0.232	0.000

Table 11.3: MAE (m/s) with the 9-predictor baseline versus the 14-predictor engineered set.

The engineered terms help the linear model, which cannot form a nonlinear density response by itself, but add nothing for the tree models.

11.4 Ridge Regularisation Sensitivity

α	0.01	0.1	1	10	100
Pooled MAE (m/s)	0.402	0.401	0.400	0.427	0.473
RMSE (m/s)	0.479	0.480	0.507	0.579	0.665

Table 11.4: Pooled out-of-fold error across the α grid (ML_05).

Mild shrinkage ($\alpha \leq 1$) leaves the MAE essentially unchanged; strong shrinkage degrades it. The grid is a sensitivity check, not a tuned optimum.

11.5 Random Forest versus XGBoost

The mean paired MAE difference (Random Forest minus XGBoost) is +0.024 m/s with a 95% family-bootstrap interval of −0.016 to +0.069 m/s. The interval spans zero, so XGBoost's lead is not reliable. Their MAE intervals overlap almost completely, and the two models should be treated as comparable.

11.6 Error Diagnostics and Feature Importance

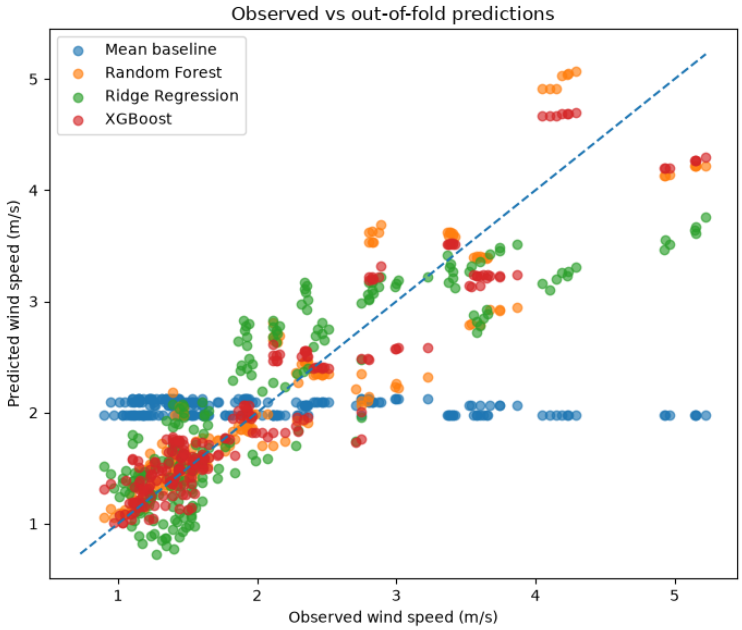


Figure 11.2: Observed versus out-of-fold predictions for all four models.

Predictions follow the 1:1 line for most cases. The largest misses occur for observed speeds above about 4 m/s, which come from the sparsest layouts and are rare in the data.

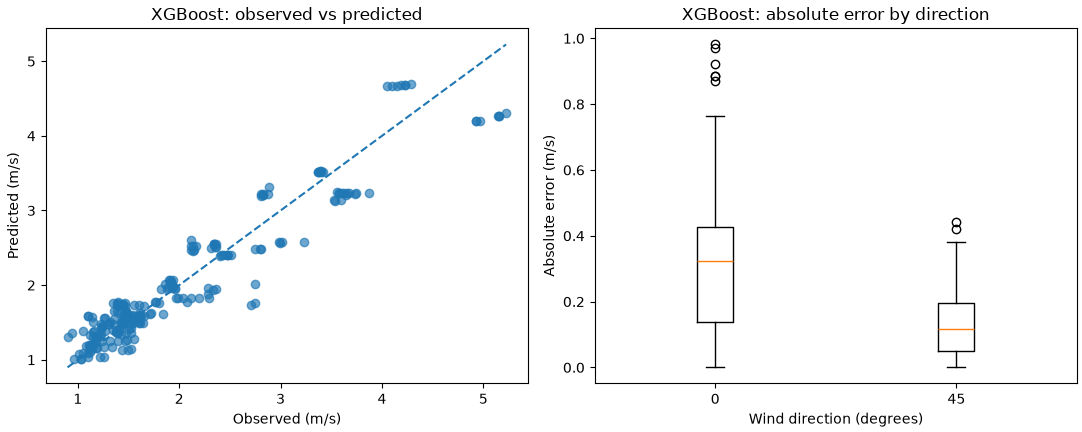


Figure 11.3: XGBoost observed versus predicted, and absolute error by wind direction (ML_06).

Absolute errors are larger at 0° (median about 0.3 m/s) than at 45° (about 0.1 m/s), consistent with the wider target spread at 0° (SD 1.19 versus 0.72 m/s).

Feature	Random Forest	XGBoost
plan_area_density	0.345	0.413
plan_area_density_sq	0.389	0.321
density_class	0.112	—
horizontal_configuration	0.041	0.083
wind_direction_sin	0.047	0.058
wind_direction_cos	0.047	0.067
height_range_m	0.004	0.016

Table 11.5: Mean importance across family folds (ML_06). "—" means outside the top ten.

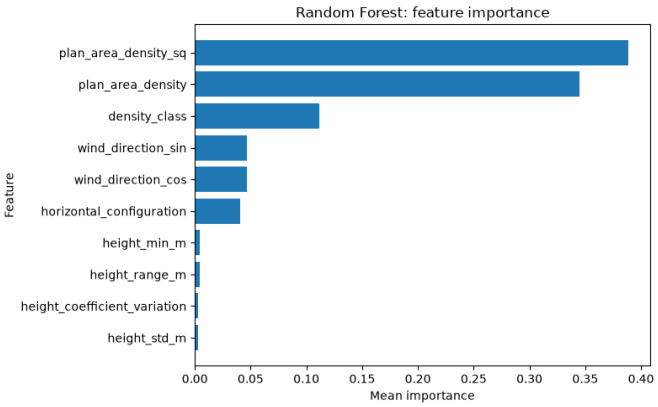


Figure 11.4: Random Forest feature importance.

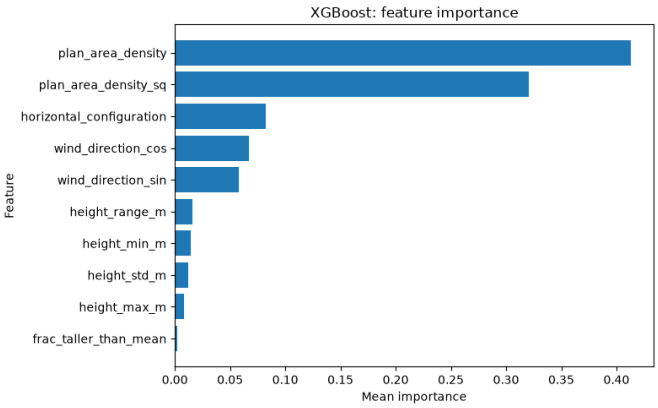


Figure 11.5: XGBoost feature importance.

Plan-area density and its square account for about 73% of importance in both models, consistent with the EDA correlation. Height statistics contribute little; wind direction contributes a small amount. Importance reflects predictive attribution within the model and is not evidence of causation.

11.7 Discussion

(1) All models with features beat the mean baseline by a wide margin, so simple geometry carries useful signal on unseen families. (2) Nonlinear trees improve clearly on Ridge (MAE 0.23–0.26

versus 0.40 m/s). (3) XGBoost and Random Forest are statistically comparable under fixed settings. (4) Validation design matters more than model choice: a layout split would overstate Random Forest by a factor of about four. (5) Engineered features help only the linear model. These conclusions apply only to this 224-case scalar dataset and the evaluated fixed settings.

12 Implementation Notes, Issues and Reproducibility

12.1 Environment

Python 3.13 in VS Code Jupyter notebooks on macOS; libraries: pandas, numpy, matplotlib, scikit-learn, xgboost, h5py and scipy.

12.2 Issues Encountered and Fixes

Issue	Cause	Resolution
Unequal ensemble sizes in ML_06 (300/200 trees) and ML_07 (100/100); XGBoost under-trained in ML_07	Settings defined separately in each notebook	Shared `RF_PARAMS`/`XGB_PARAMS`; both notebooks now give identical family-held-out metrics
Categorical importance understated	Importance averaged over one-hot columns	Sum one-hot columns per parent feature within each fold, then average across folds
`Axes.boxplot()` unexpected argument `labels`	Newer Matplotlib renamed it	Use `tick_labels=` in boxplot calls
Download page stuck on "Loading…"	Dynamic page	Page content copied manually; 224 case folders confirmed
Mixed NetCDF formats	NetCDF4/HDF5 and classic NetCDF in one archive	`h5py` first, fall back to `scipy.io.netcdf_file`

Table 12.1: Issues encountered during implementation.

12.3 Run Order and Outputs

Notebook	Purpose	Key outputs
ML_00	Literature and problem framing	—
ML_01	Raw audit, join, dictionary	`layout_level_dataset.csv`, data dictionary, archive manifests
ML_02	EDA and target definition	Plots A–F
ML_03	Features and leakage audit	`feature_engineered_dataset.csv`
ML_04	Grouped splits, preprocessing	`family_fold_manifest.csv`
ML_05	Ridge regression	`ridge_*.csv`
ML_06	Random Forest, XGBoost	`tree_*.csv`, importance tables
ML_07	Comparison and uncertainty	`model_comparison.csv`, `metrics_all_protocols.csv`, bootstrap and paired tables

Table 12.2: Notebook run order and outputs.

Run ML_01 to ML_07 in order from the repository root. Results files are runtime outputs and are never read as inputs by later notebooks.

13 Limitations

- **Scalar target:** one value per case; no wake patterns, street-canyon effects or spatial variation.
- **Small sample:** 224 cases, 112 layouts and only 16 families, so fold scores vary and intervals are wide.
- **Two directions only:** 0° and 45°; conclusions about arbitrary angles are not justified.
- **Constant mean height:** all cases share 16 m, so height effects are limited to dispersion statistics.
- **Wind only:** no temperature or heat claims.
- **Fixed hyperparameters:** no nested tuning, so rankings may shift; the Random Forest/XGBoost comparison is inconclusive.
- **Archive coverage:** profile, topology and NetCDF archives are samples (32 members each).
- **Idealised simulations:** the study concerns surrogate accuracy on idealized cases, not safety-critical design approval.

14 Conclusion and Phase 2 Outlook

A reproducible Phase 1 pipeline runs from raw UrbanTALES files to a validated 224-case layout-level table, with archive auditing, exploratory analysis, feature engineering, leakage controls, grouped validation and a comparison of a mean baseline, Ridge, Random Forest and XGBoost. Under family-held-out validation XGBoost reaches an MAE of 0.232 m/s ($R^2 = 0.903$), a 71% reduction against the mean baseline, though it is statistically comparable to Random Forest (0.256 m/s). Plan-area density dominates the predictions, and a layout-level split would overstate performance by about a factor of four.

The scalar baseline sets the reference that Phase 2 must beat. Phase 2 will use the sampled topology grids and NetCDF fields as the starting point for a rasterised-layout U-Net or conditional CNN predicting velocity fields, evaluated on held-out layouts with field MAE, normalised RMSE and error maps. This is a plan, not an implemented result.

15 Team Contributions

Team member	Notebook contribution
Suhaani Garg — 2401010462	Notebooks 1 and 7 — ML_01: raw-data audit, data extraction, processed dataset and data dictionary; ML_07: evaluation, model comparison and uncertainty
Aarya Srivastava — 2401010008	Notebook 2 — ML_02: exploratory data analysis and target understanding
Jashvita Lakshmi Omkaram — 2401010310	Notebooks 3–4 — ML_03 feature engineering and leakage audit; ML_04 grouped validation and preprocessing
Palak — 2401010042	Notebook 5 — ML_05: Ridge regression baseline and regularisation analysis
Samarth Sangtani — 2401010408	Notebook 6 — ML_06: Random Forest and XGBoost

Table 15.1: Notebook ownership by team member.

All members should be prepared to explain the complete pipeline and the decisions in their assigned notebooks.

References

[1] I. Goodfellow et al., "Generative Adversarial Nets," *Advances in Neural Information Processing Systems*, vol. 27, 2014.

[2] A. Farimani et al., "Deep Learning for Wall Bounded Turbulent Flows," 2017. (Title and year as given in the project brief; verify full publication details against the faculty source.)

[3] S. L. Brunton, B. R. Noack and P. Koumoutsakos, "Machine Learning for Fluid Mechanics," *Annual Review of Fluid Mechanics*, vol. 52, pp. 477–508, 2020, doi: 10.1146/annurev-fluid-010719-060214.

[4] UrbanTALES dataset (idealized building arrays), documentation and datasheet supplied with the project (CC BY-NC 4.0 per the supplied extraction notes).

A Data Dictionary of the Modelling Table

Field	Units	Meaning	Model use
simulation_id	—	Case identifier (layout + direction)	Traceability
layout_id	—	Base geometry shared by its direction runs	Grouping only
family_id	—	Arrangement letter + density code (16)	Primary group
target_U_canopy_mean_m_s	m/s	Canopy-averaged wind speed	Target
plan_area_density	ratio	Plan-area density	Numeric
plan_area_density_sq	ratio^2	Squared density	Numeric
height_mean_m	m	Mean height (constant 16 m)	Not a predictor
height_std_m	m	Std. dev. of heights	Numeric
height_max/min/range_m	m	Max, min, range	Numeric
height_skew	—	Skewness of heights	Numeric
frac_taller_than_mean	fraction	Share above mean	Numeric
wind_direction_deg	deg	Inflow direction (0 or 45)	Replaced by sin/cos
wind_direction_sin/cos	—	Circular encoding	Numeric
height_coefficient_variation	—	height_std / height_mean	Numeric
configuration	category	uniform, CL, HR, SM	One-hot
configuration_metadata_label	category	Original catalogue label	Excluded
horizontal_configuration	category	Aligned or staggered	One-hot
density_class	category	Sparse, medium, dense	One-hot
building_count	count	Buildings in the height list	Ablation only

Table A.1: Fields of the modelling table.